

Experiment 1

Arman Imam

EEE3342C-25Fall 0011

9/8/2025

Equipment:

Computer with Xilinx ISE software, Vivado 2017.4 specifically, and Basys3 Board, which is used to test multiple logic gates.

Objective:

The objective of experiment 1 is to introduce the user to Vivado, and to understand how to use 2-input AND, OR, XOR logic gates, inverter, and two-input and five output circuit.

Design Steps:

I began by creating a folder for each part (for part 2, I've created a project for each gate).

Then for each gate, I created a source and sim file in order to find out how the logic reacts to two inputs and one output for parts 1 and 2. I went ahead to find the logic specifications, as well as the waveform for information. For part 3, I did the same, but I used 2 inputs and 5 outputs,

- “`timescale 1ns /1ps” at the top of each code

Simulation code used (up until NOT Gate):

```

module Part_2_Sim();
    reg Inp_1, Inp_2;
    wire Outp;

    Lab1_part2 UUT(.Inp_1(Inp_1), .Inp_2(Inp_2),
    .Outp(Outp));

    initial
    begin
        Inp_1 = 0;
        Inp_2 = 0;
        #10
        Inp_1 = 1;
        Inp_2 = 0;
        #10
        Inp_1 = 0;
        Inp_2 = 1;
        #10
        Inp_1 = 1;
        Inp_2 = 1;
    end
end

```

Part 1: AND Gate Schematic

```

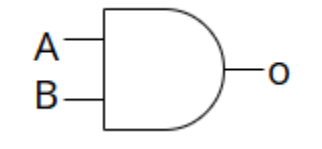
} module Lab1_part1(
    input Inp_1, Inp_2,
    output Outp
);

    assign Outp = Inp_1 & Inp_2;
} endmodule

```

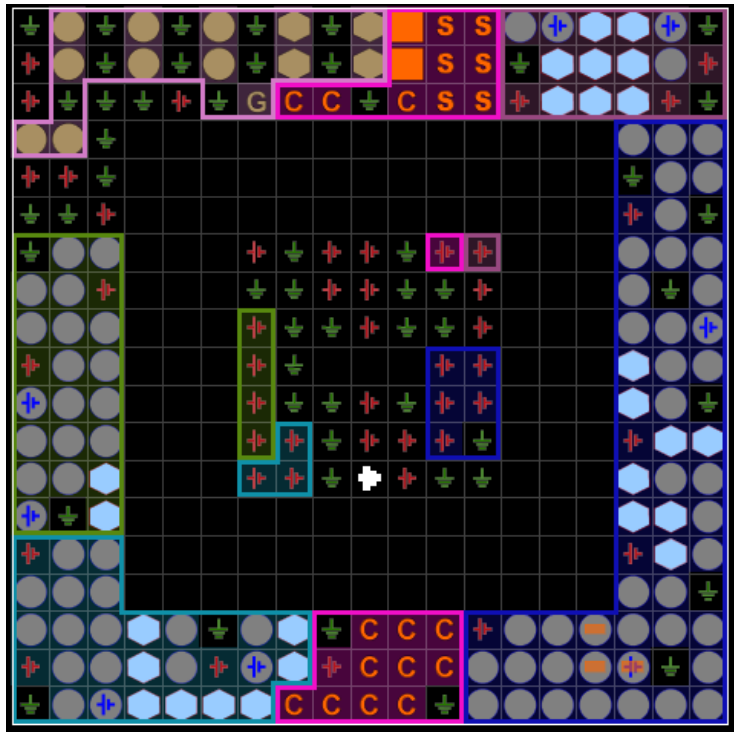
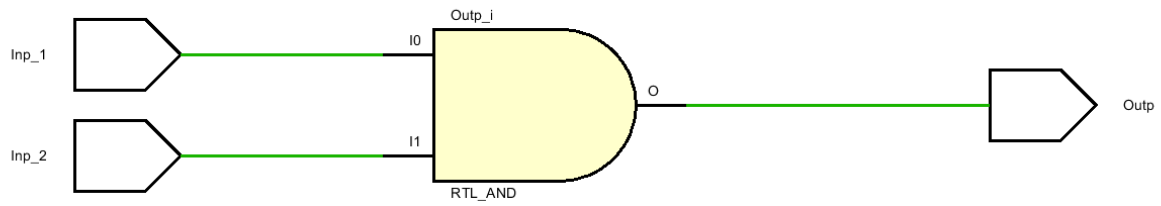
Logic Diagram:

The results from the Basys 3 board testing agreed with the simulation expected for an AND Gate.

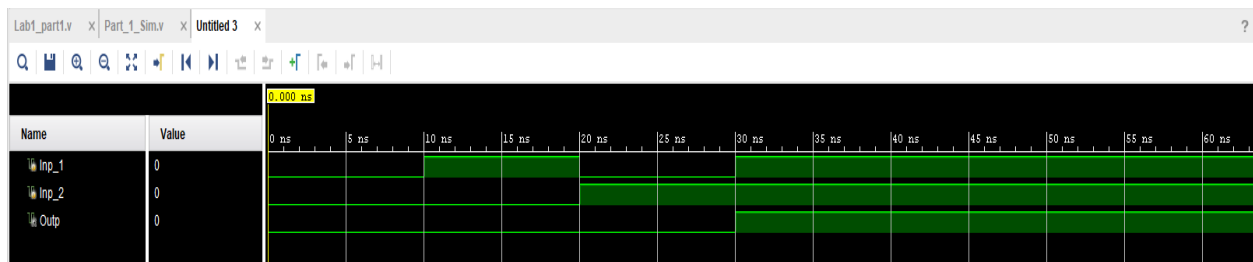


Design Specification Plan:

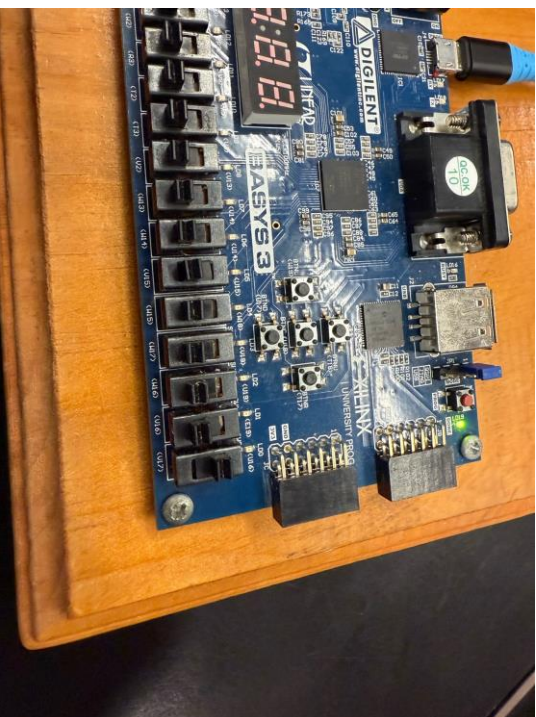
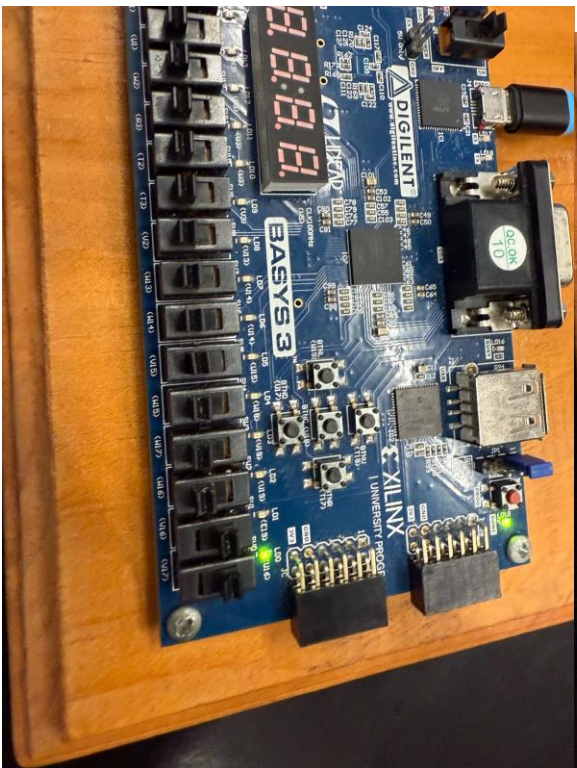
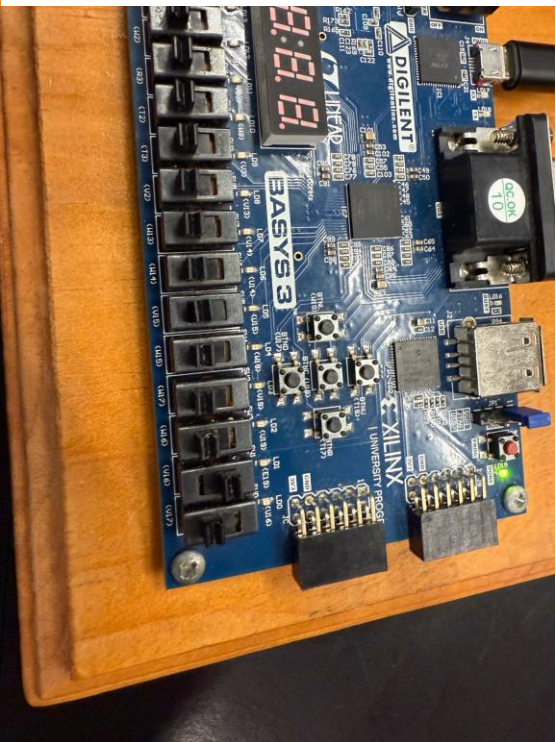
The experiment calls for an AND logic gate, with two inputs (A and B), and one output(O)



Results Statement:



SW0	SW1	LED0
0	0	0
0	1	0
1	0	0
1	1	1

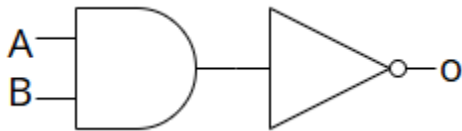


Part 2 NAND Gate Schematic:

```
module Lab1_part2(  
    input Inp_1, Inp_2,  
    output Outp  
);  
  
    assign Outp = ~(Inp_1 & Inp_2);  
endmodule
```

Logic Diagram:

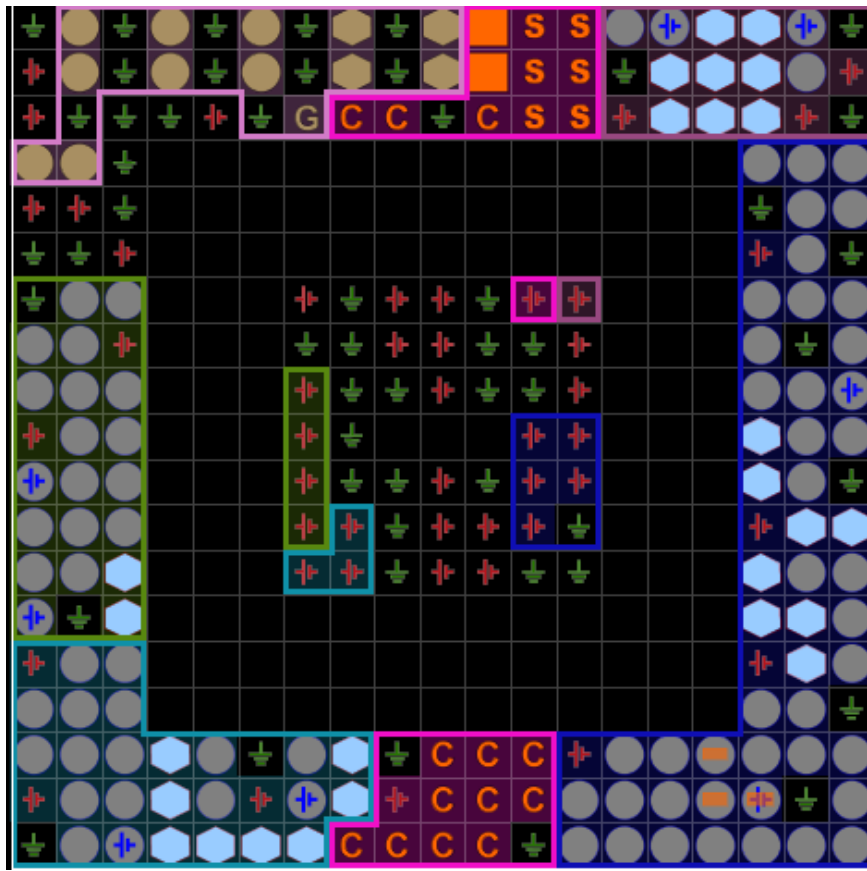
The results from the Basys 3 board testing agreed with the simulation expected for an NAND Gate.



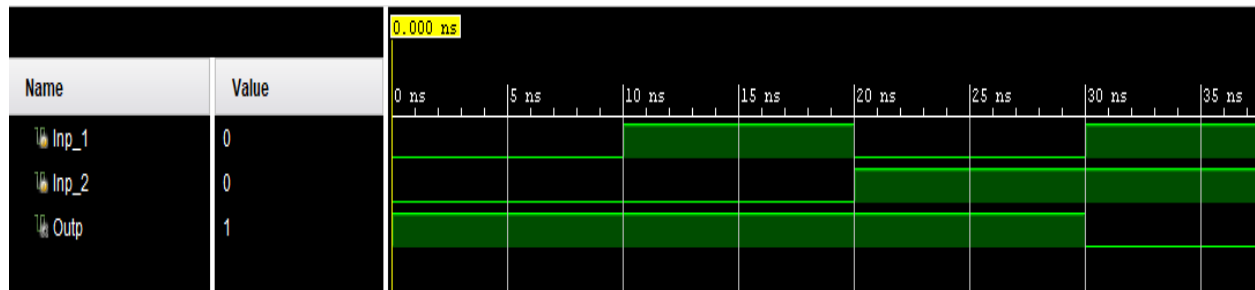
Design Specification Plan:

The experiment calls for a NAND logic gate, with two inputs (A and B), and one output(O)

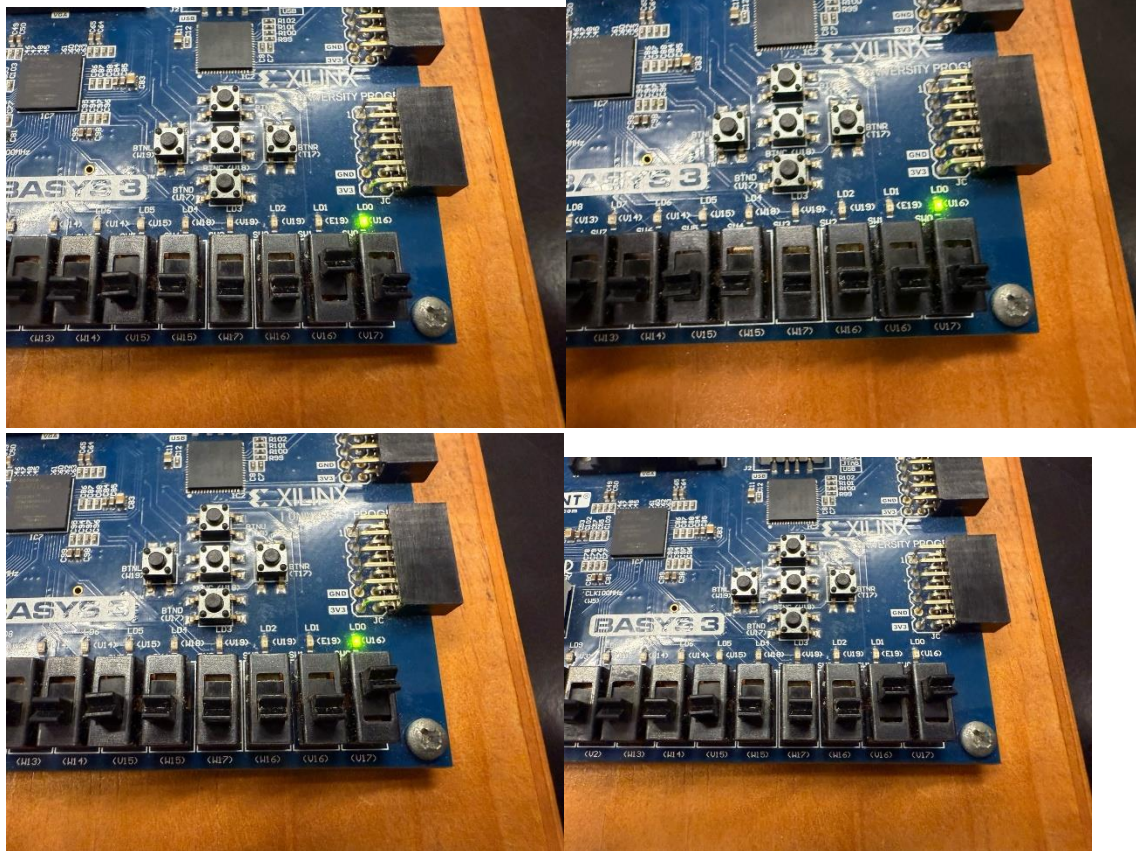




Results Statement:



SW0	SW1	LED0
0	0	1
0	1	1
1	0	1
1	1	0



Part 2 OR Gate Schematic:

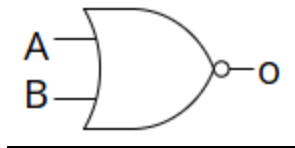
```

2 |
3 | module Lab1_part2(
4 |     input Inp_1, Inp_2,
5 |     output Outp
6 | );
7 |
8 |     assign Outp = Inp_1 | Inp_2;
9 | endmodule
10 |

```

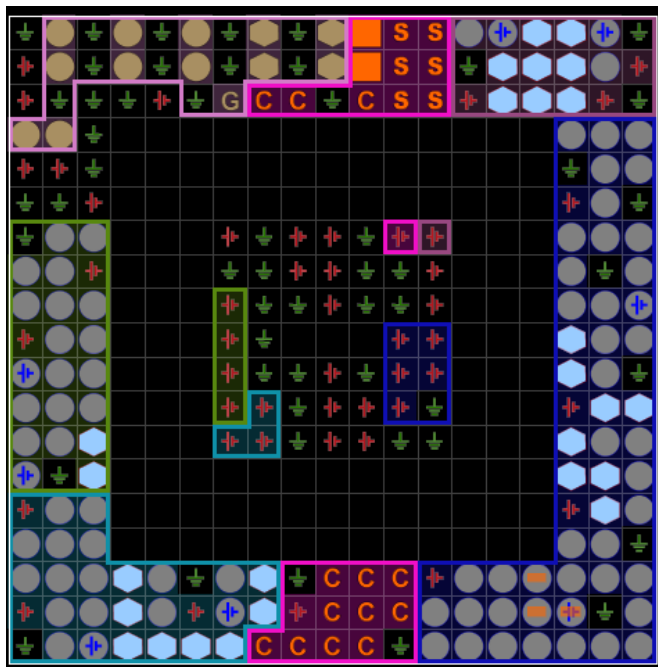
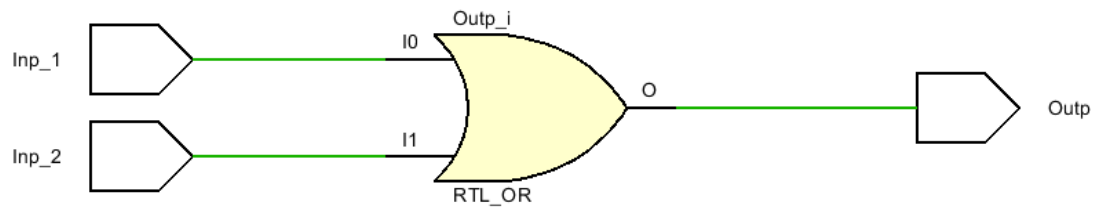
Logic Diagram:

The results from the Basys 3 board testing agreed with the simulation expected for an OR Gate.

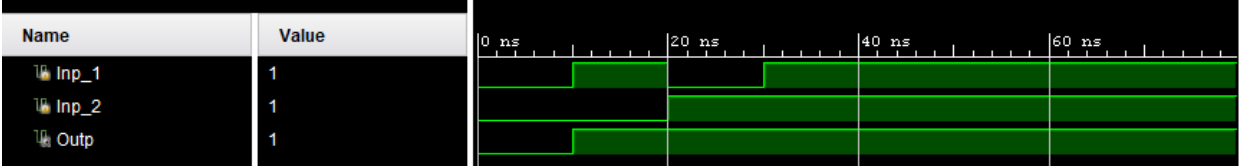


Design Specification Plan:

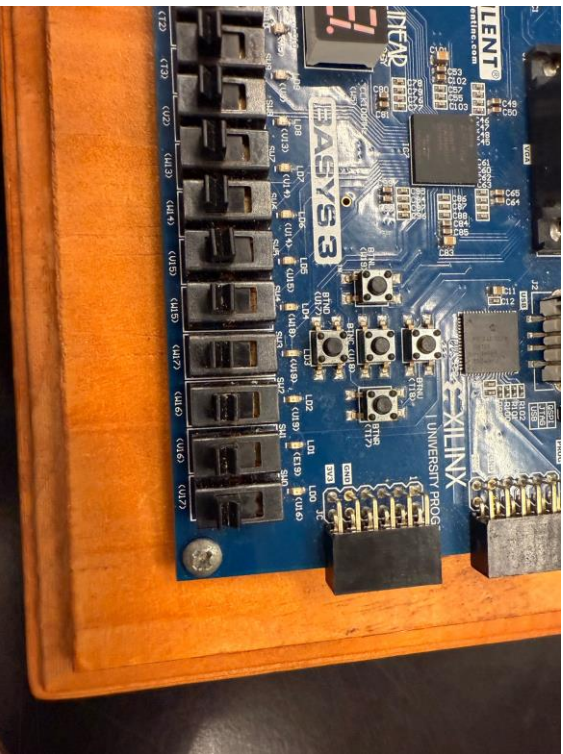
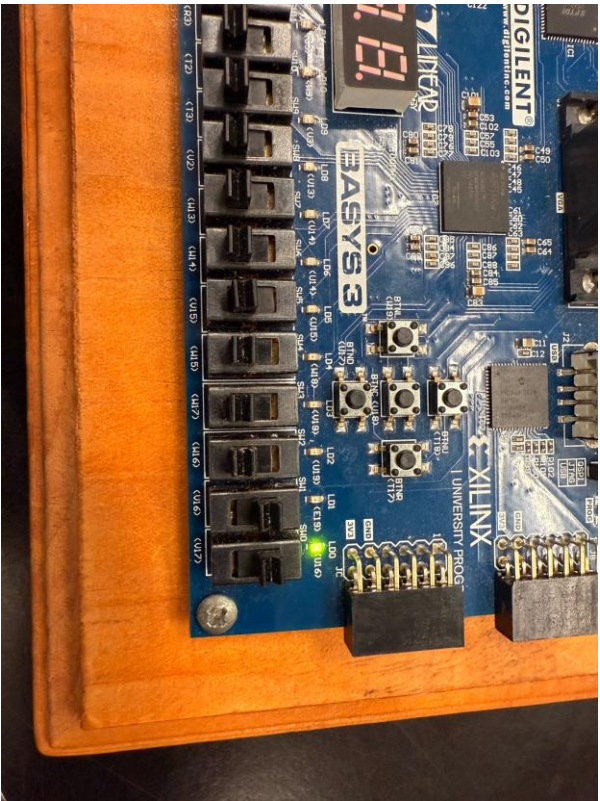
The experiment calls for an OR logic gate, with two inputs (A and B), and one output(O)



Results Statement:



SW0	SW1	LED0
0	0	0
0	1	1
1	0	1
1	1	1

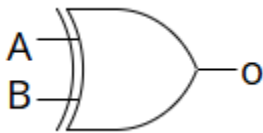


Part 2 XOR Gate Schematic:

```
module Lab1_part2(  
    input Inp_1, Inp_2,  
    output Outp  
);  
  
    assign Outp = Inp_1 ^ Inp_2;  
endmodule
```

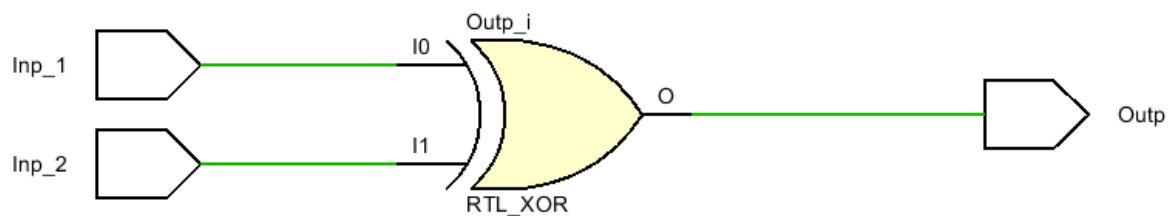
Logic Diagram:

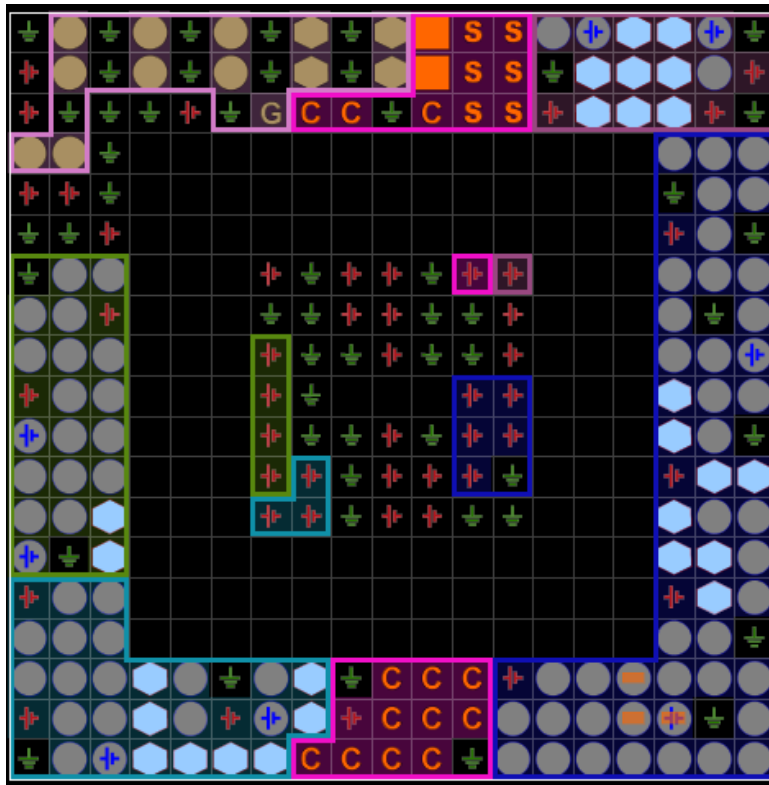
The results from the Basys 3 board testing agreed with the simulation expected for an XOR Gate.



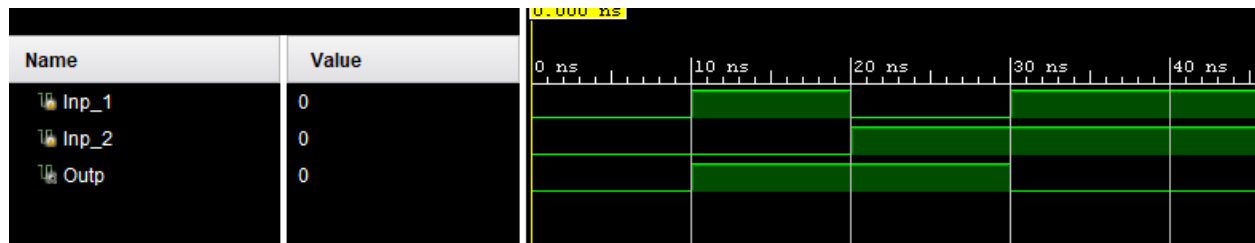
Design Specification Plan:

The experiment calls for an XOR logic gate, with two inputs (A and B), and one output(O)

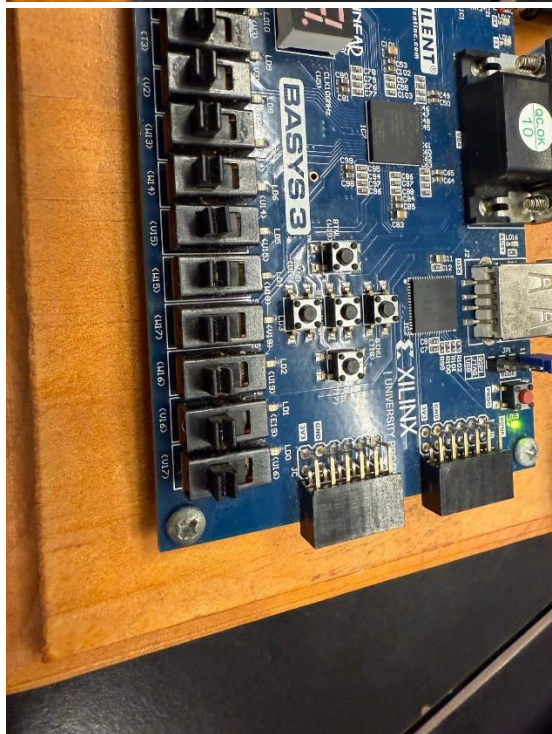
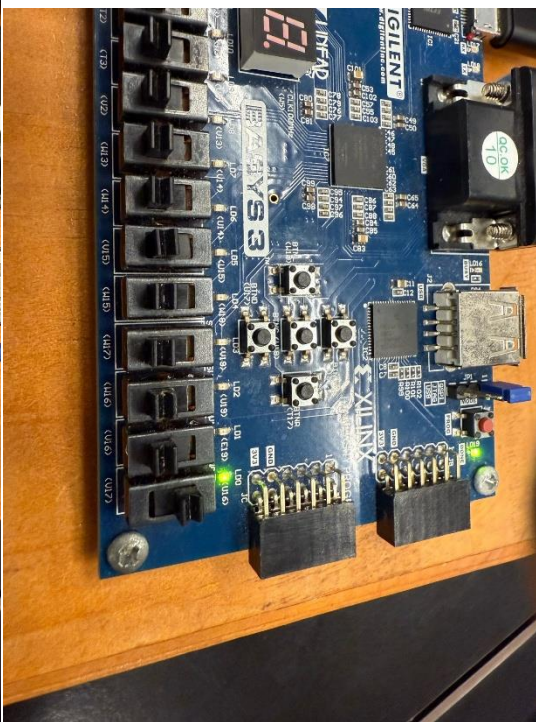
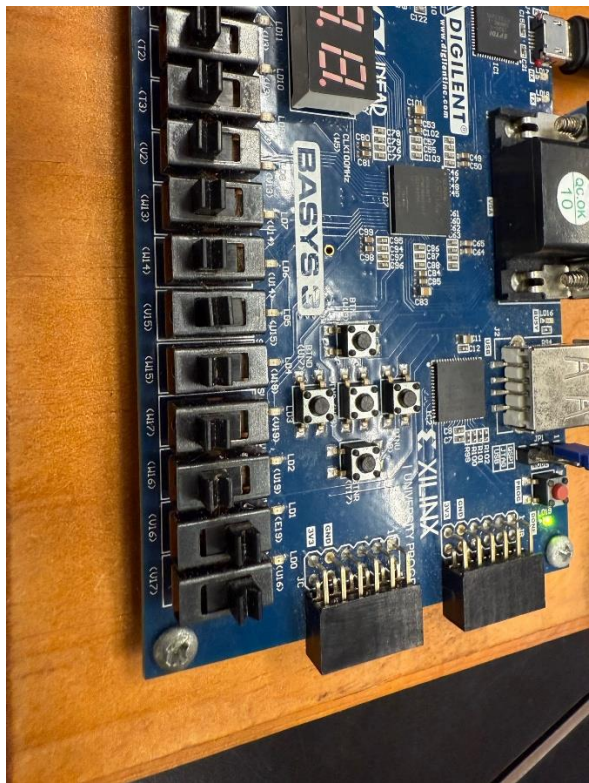
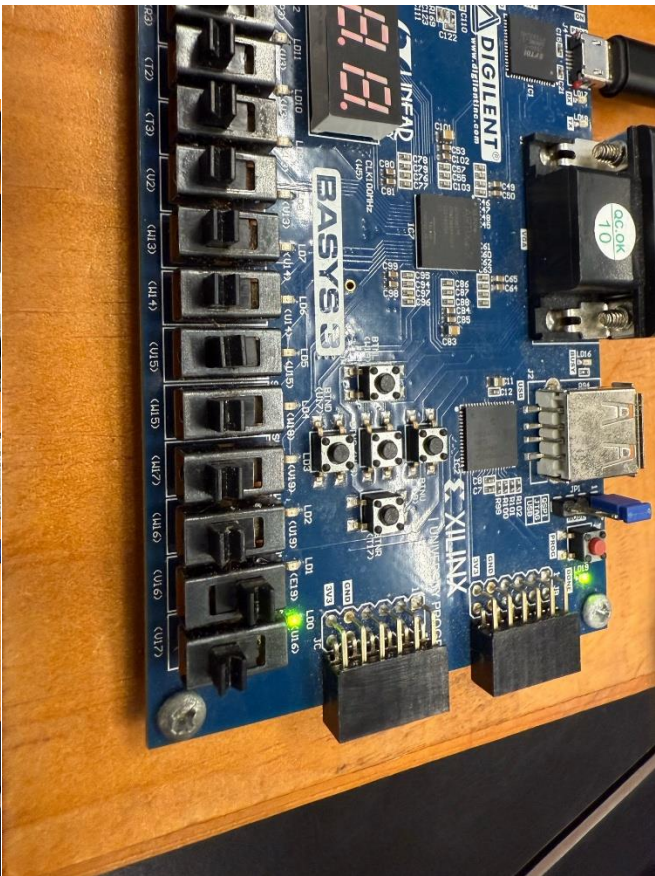




Results Statement:



SW0	SW1	LED0
0	0	0
0	1	1
1	0	1
1	1	0

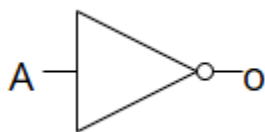


Part 2 NOT Gate Schematic:

```
1 module Lab1_part2(  
2     input Inp_1,  
3     output Outp  
4 );  
5  
6     assign Outp = ~Inp_1;  
7 endmodule  
8  
9  
10  
11  
12 module Part_2_Sim();  
13     reg Inp_1;  
14     wire Outp;  
15  
16     Lab1_part2 UUT(.Inp_1(Inp_1),  
17     .Outp(Outp));  
18  
19     initial  
20     begin  
21         Inp_1 = 0;  
22         #10  
23         Inp_1 = 1;  
24     end  
25  
26 endmodule  
27
```

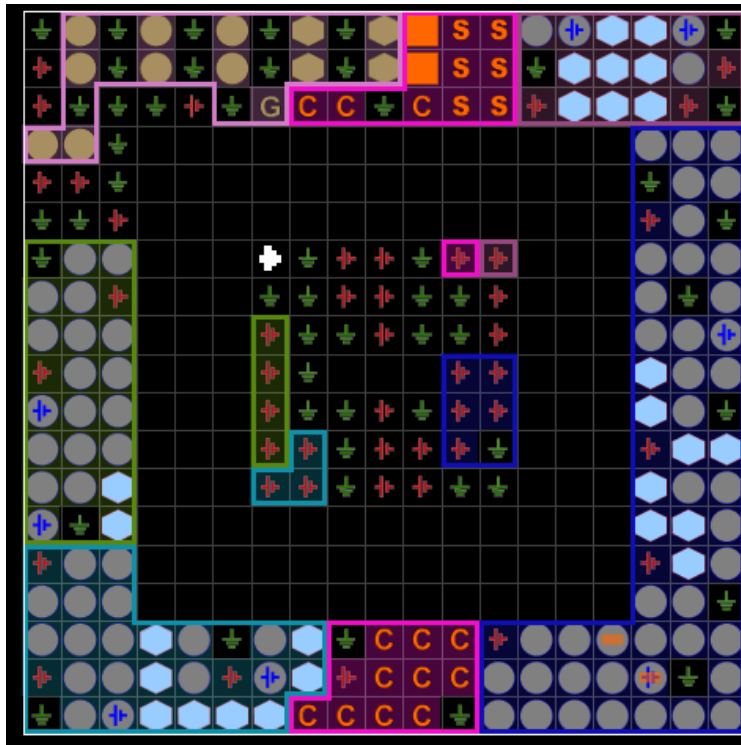
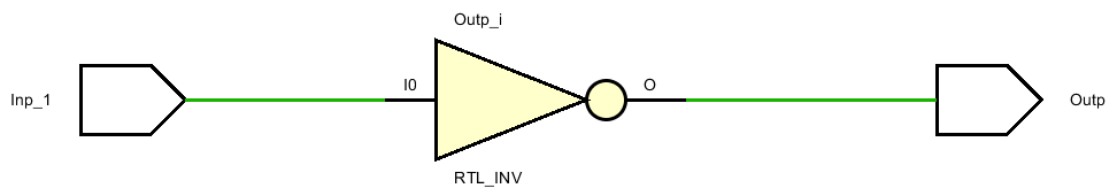
Logic Diagram:

The results from the Basys 3 board testing agreed with the simulation expected for a NOT Gate.

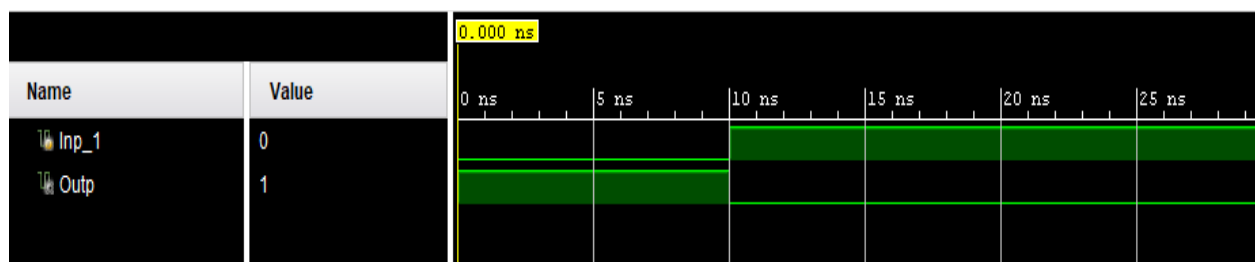


Design Specification Plan:

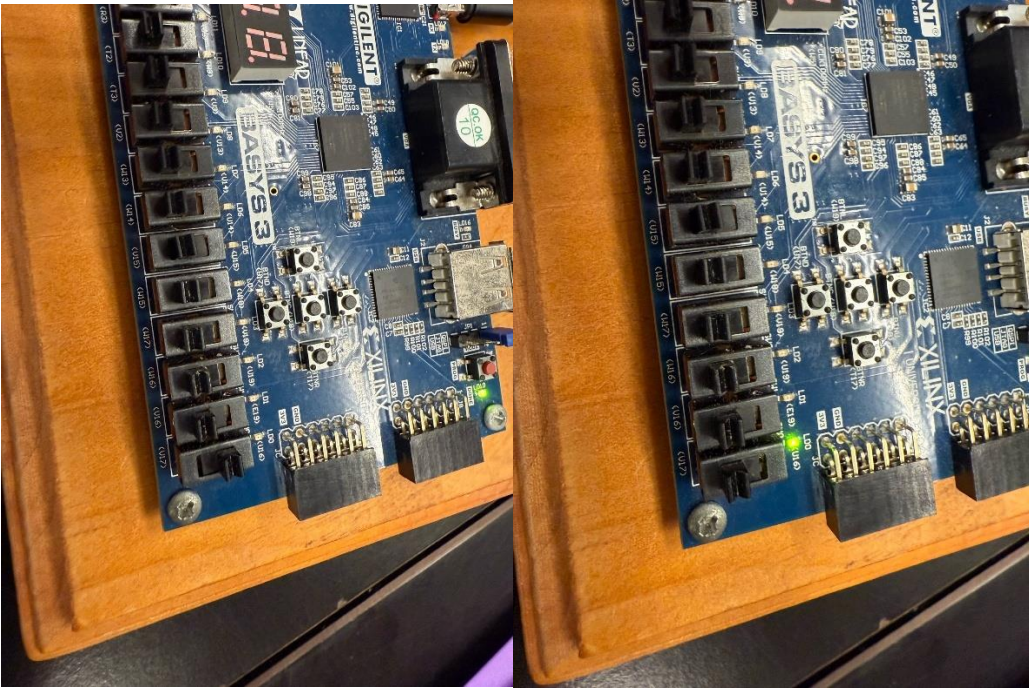
The experiment calls for a NOT logic gate, with one input (A), and one output(O)



Results Statement:



SW0	LED0
0	1
1	0



Part 3 Gate Schematic:

Let

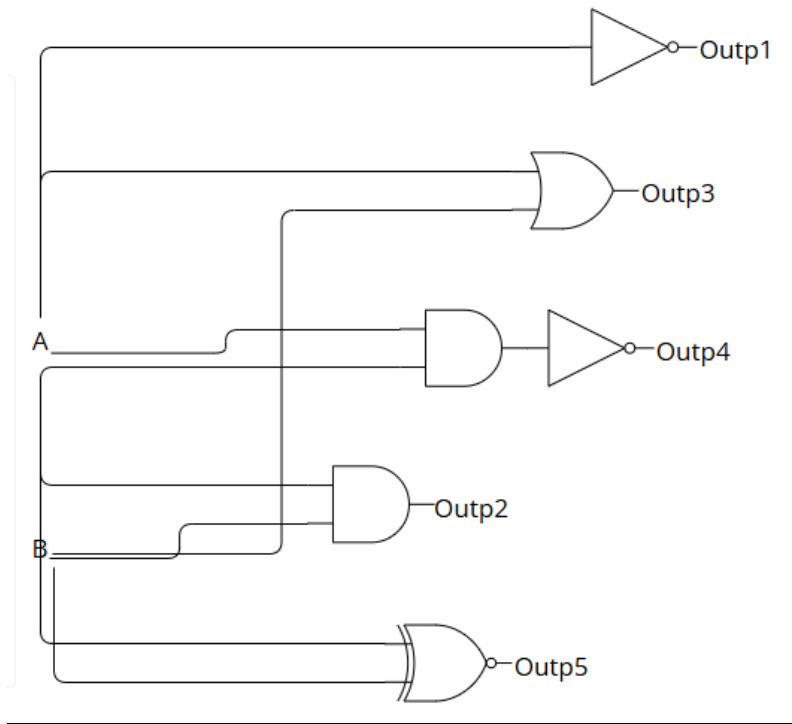
- LED0(S) be outp1,
- LED0(W) be outp2
- LED0(X) be outp3
- LED0(Y) be outp4
- LED0(Z) be outp5

```
23 module Lab1_part3(  
24     input Inp1, Inp2,  
25     output Outp1, Outp2, Outp3, Outp4, Outp5  
26 );  
27  
28     assign Outp1 = ~Inp1;  
29     assign Outp2 = Inp1 & Inp2;  
30     assign Outp3 = Inp1 | Inp2;  
31     assign Outp4 = ~(Inp1 & Inp2);  
32     assign Outp5 = Inp1 ^ Inp2;  
33  
34 endmodule
```

```
module Lab3_Sim();  
    reg Inp1, Inp2;  
    wire Outp1, Outp2, Outp3, Outp4, Outp5;  
  
    Lab1_part3 UUT(  
        .Inp1(Inp1),  
        .Inp2(Inp2),  
        .Outp1(Outp1),  
        .Outp2(Outp2),  
        .Outp3(Outp3),  
        .Outp4(Outp4),  
        .Outp5(Outp5));  
  
    initial  
    begin  
        Inp1 = 0;  
        Inp2 = 0;  
        #10  
        Inp1 = 1;  
        Inp2 = 0;  
        #10  
        Inp1 = 0;  
        Inp2 = 1;  
        #10  
        Inp1 = 1;  
        Inp2 = 1;  
    end  
endmodule
```

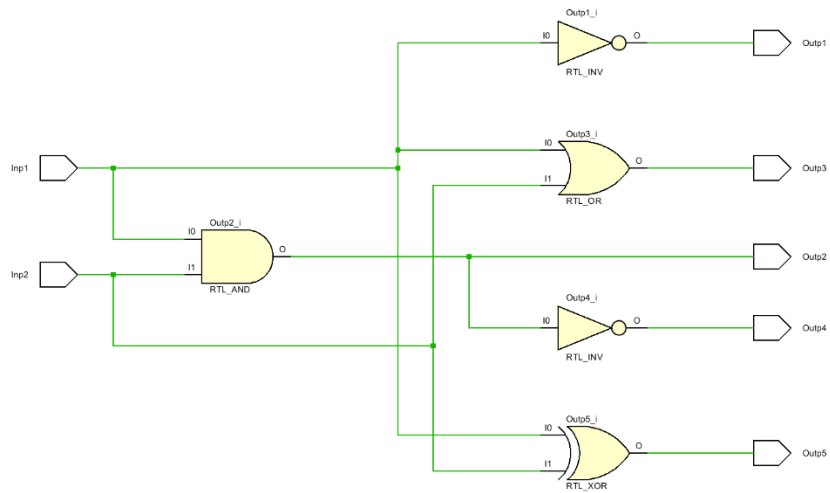
Logic Diagram:

The results from the Basys 3 board testing agreed with the simulation expected for a NOT, AND, OR, NAND, and XOR logic.

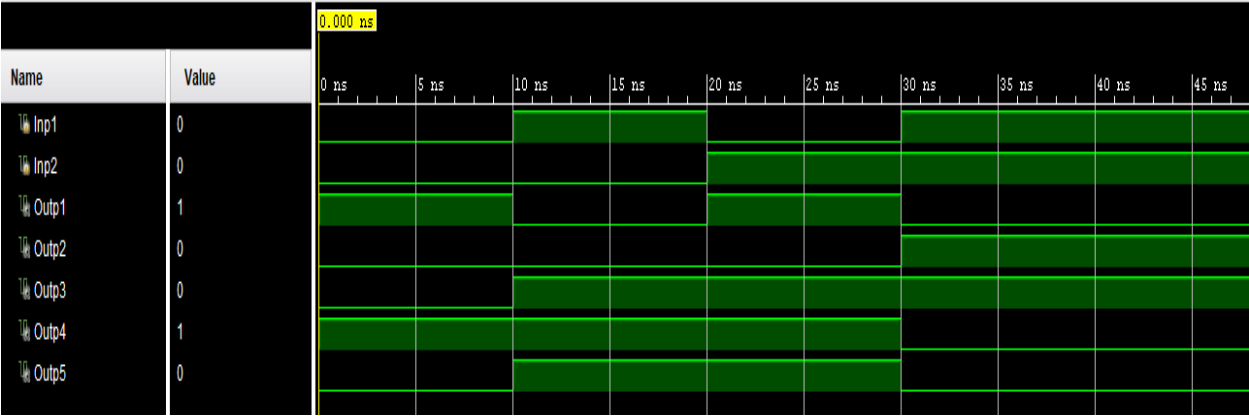


Design Specification Plan:

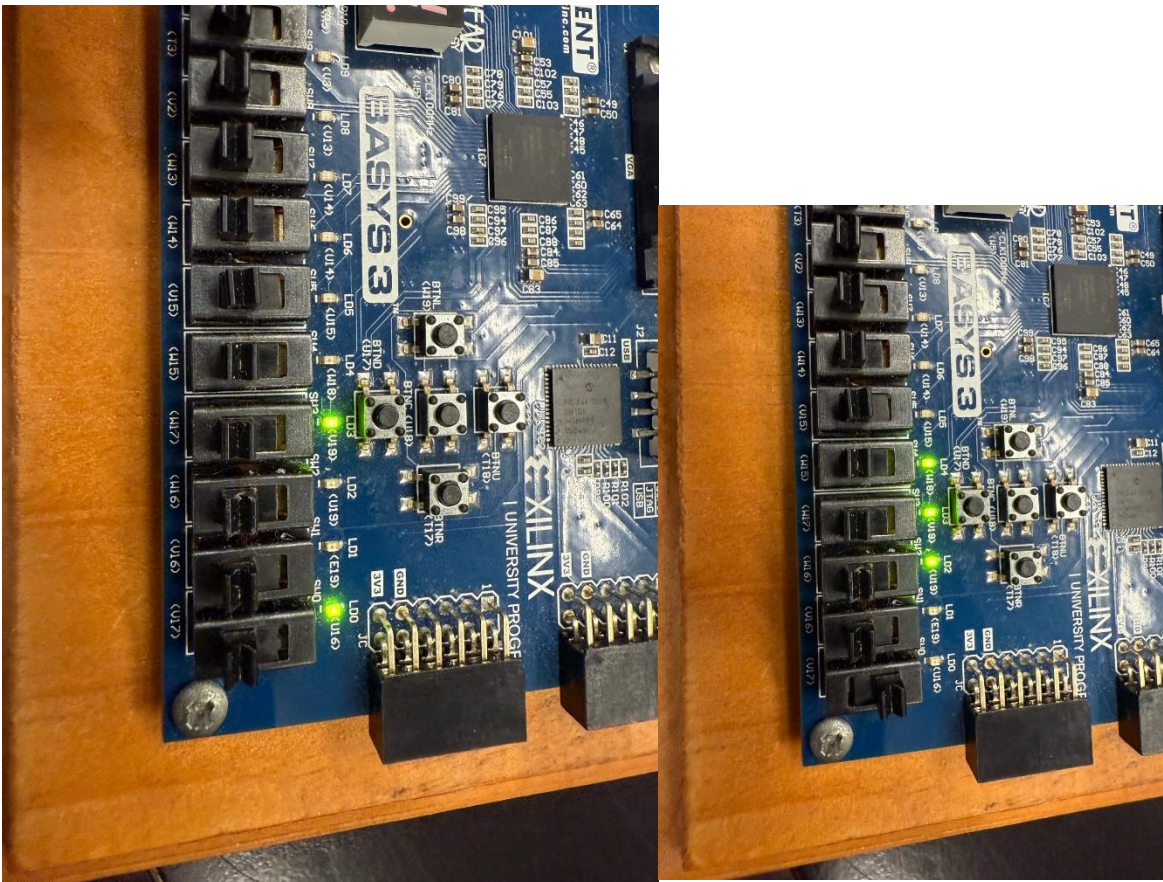
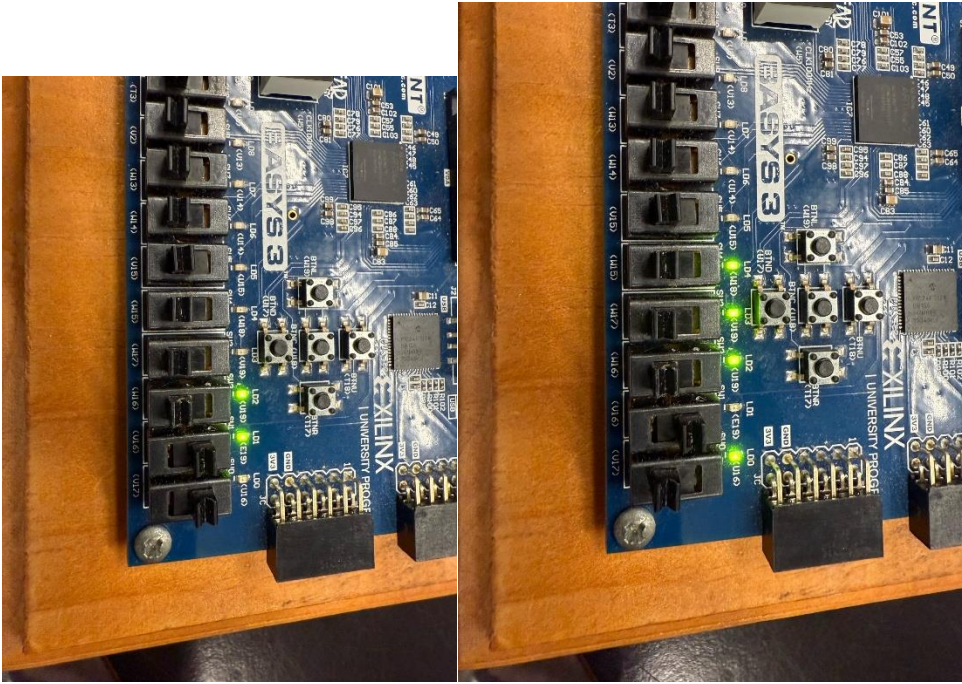
The experiment calls for a NOT, AND, NAND, OR, and XOR logic gate, with two inputs (A and B), and 5 output(Outp1, Outp2, Outp3, Outp4, Outp5)



Results Statement:



A	B	LED0(S)	LED1(W)	LED2(X)	LED3(Y)	LED4(Z)
0	0	1	0	0	1	0
1	0	0	0	1	1	1
0	1	1	0	1	1	1
1	1	0	1	1	0	0



Test Plan:

N/A

Conclusion:

This portion of the experiment was a success. The logic gates and testing through Vivado have provided the same results. The use of the Baysys3 motherboard also provided confirmation of the logic gates results. Testing logic gates through this software helps further understanding of logic gates and how they can be implemented